Humanoid robotics and biomimetic systems

Perception – object detection and pose estimation

Doc. dr. sc. Filip Šuligoj

fsuligoj@fsb.hr

University of Zagreb

fsb University of Zagreb Faculty of Mechanical Engineering and Naval Architecture

Object detection and pose estimation

Object detection (2D)

Identifies and localizes objects in an image by assigning class labels and estimating their positions in image coordinates (e.g., bounding boxes or segmentation masks).

Object detection (3D)

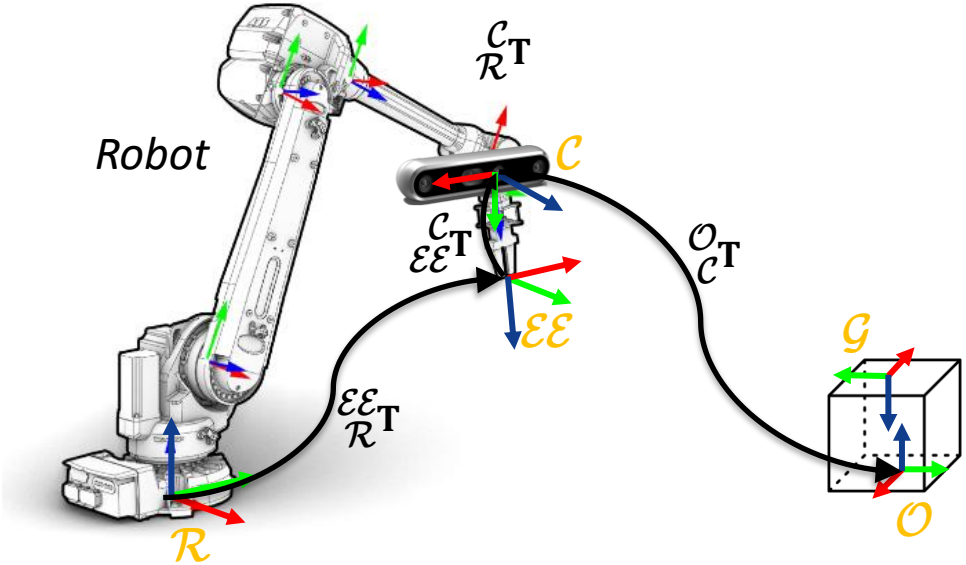
Identifies and localizes objects in 3D space by estimating their position and extent in a spatial reference frame (e.g., camera or world), typically using point clouds or depth data to produce 3D bounding boxes or object clusters.

Pose estimation *(objective definition) determines the 6DoF pose (x,y,z,;roll,pitch,yaw) of a known object in a reference frame (e.g., camera or robot base), typically using sensor data and prior knowledge; it may rely on detection, learned features, or geometric cues without requiring full point-wise alignment.*

Registration (methodological approach) computes the rigid transformation between two representations *(e.g., CAD model ↔ observed point cloud) by explicitly aligning geometry (e.g., ICP, feature-based matching), yielding the same 6DoF pose as the transformation that minimizes spatial discrepancy.*

- ***Key distinction****: pose estimation is the problem formulation (recover object pose), while registration is a geometric solution class to that problem; in practice, modern pipelines combine both (e.g., detection → segmentation → registration-based refinement).*

Eye-in-hand robot configuration



$\mathcal{R}$ - robot base coordinate system
$\mathcal{W}$ - world coordinate system
$\mathcal{C}$ - camera coordinate system
$\mathcal{F}$ - flange coordinate system
$\mathcal{EE}$ - end effector coordinate system
$\mathcal{O}$ - object coordinate system
$\mathcal{G}$ - grasp pose coordinate system

In an eye-in-hand configuration (camera rigidly mounted on the robot end-effector):

- *The camera is attached to the robot tool, so its pose changes with robot motion ($^{\mathcal{EE}}_{\mathcal{R}}\mathbf{T}$).*
- *The $\mathcal{C}$ and the $\mathcal{EE}$ relation is defined by transformation $^{\mathcal{C}}_{\mathcal{EE}}\mathbf{T}$, obtained from calibration.*
- *The point cloud registration (e.g. ICP) is used to localize the object $\mathcal{O}$ in camera coordinate system $\mathcal{C}$, yielding the transformation $^{\mathcal{O}}_{\mathcal{C}}\mathbf{T}$.*
- *The **grasp pose** is defined relative to the object: $^{\mathcal{G}}_{\mathcal{O}}\mathbf{T}$*

Eye-in-hand calibration

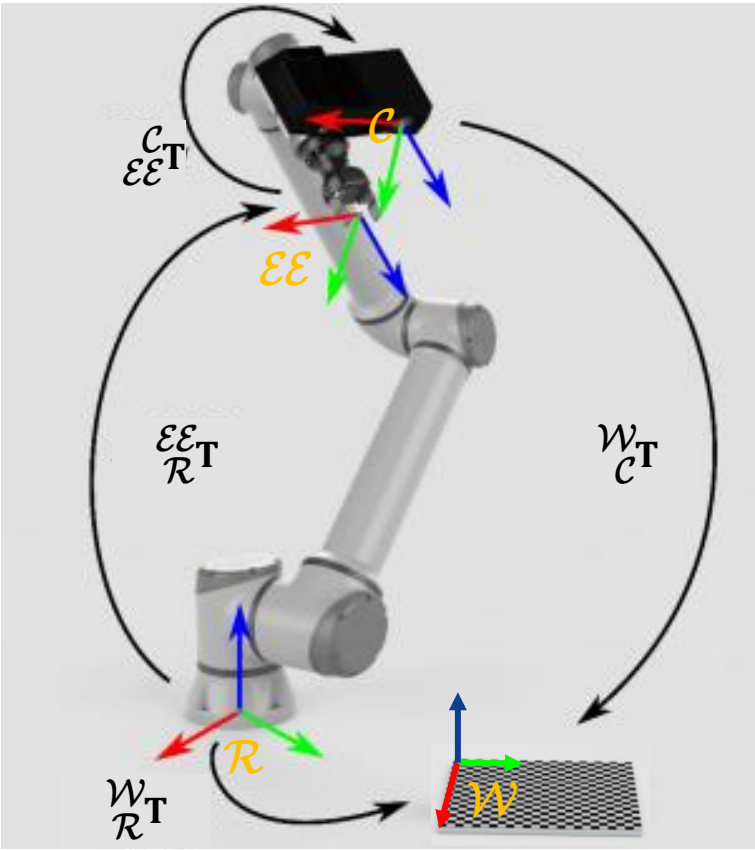
$$ {}^{\mathcal{W}}_{\mathcal{R}}\mathbf{T} = {}^{\mathcal{E}\mathcal{E}}_{\mathcal{R}}\mathbf{T}_1 \, {}^{\mathcal{C}}_{\mathcal{E}\mathcal{E}}\mathbf{T} \, {}^{\mathcal{W}}_{\mathcal{C}}\mathbf{T}_1 = {}^{\mathcal{E}\mathcal{E}}_{\mathcal{R}}\mathbf{T}_2 \, {}^{\mathcal{C}}_{\mathcal{E}\mathcal{E}}\mathbf{T} \, {}^{\mathcal{W}}_{\mathcal{C}}\mathbf{T}_2 \ldots\ldots $$

Constant

$$ \boldsymbol{A_i X} = \boldsymbol{X B_i} $$

Known pose of robot tool between positions

Fixed calibration board observed from multiple robot poses to estimate the camera–world transform.

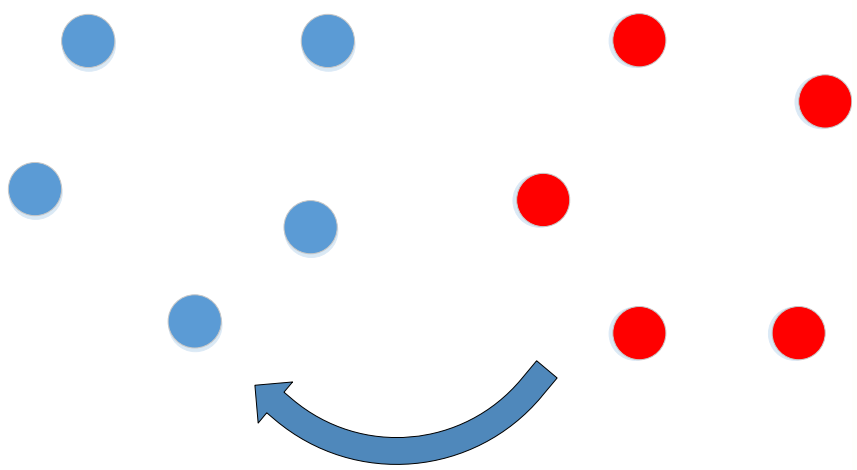


Solution to $\boldsymbol{A_i X} = \boldsymbol{X B_i}$ problem:

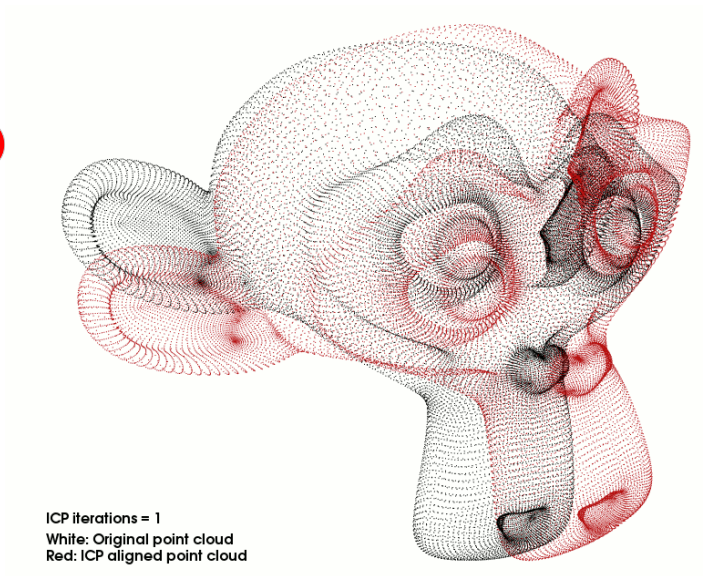
- *R. Tsai, R. Lenz A New Technique for Fully Autonomous and Efficient 3D Robotics Hand/EyeCalibration*
- *F. Park, B. Martin Robot Sensor Calibration: Solving AX = XB on the Euclidean Group*
- *R. Horaud, F. Dornaika Hand-Eye Calibration*
- *N. Andreff, R. Horaud, B. Espiau On-line Hand-Eye Calibration*
- *K. Daniilidis Hand-Eye Calibration Using Dual Quaternions*

Registration

- ***Registration refers to the process of establishing a spatial correspondence between two sets of data, enabling their alignment within a common coordinate system.***
- *This fundamental technique is pivotal across various domains including machine vision, robotics, data fusion, object recognition, navigation, and medical imaging.*
- *The datasets involved typically consist of point sets located in a finite-dimensional real vector space, commonly in two or three dimensions.*
- *The primary goal of registration is to optimally align these two point sets by estimating the most accurate transformation needed to achieve alignment.*

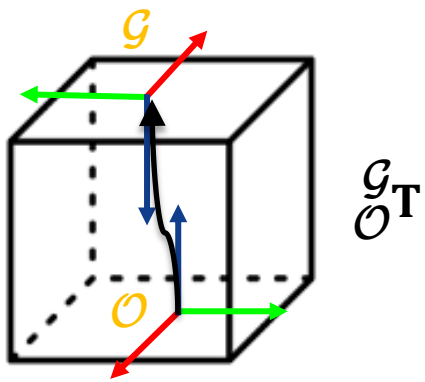


TRANSFORMATION

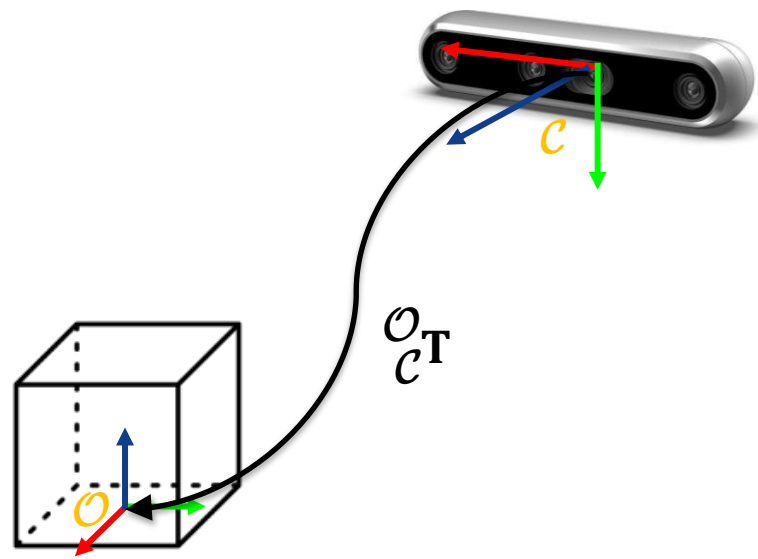


Point cloud registration

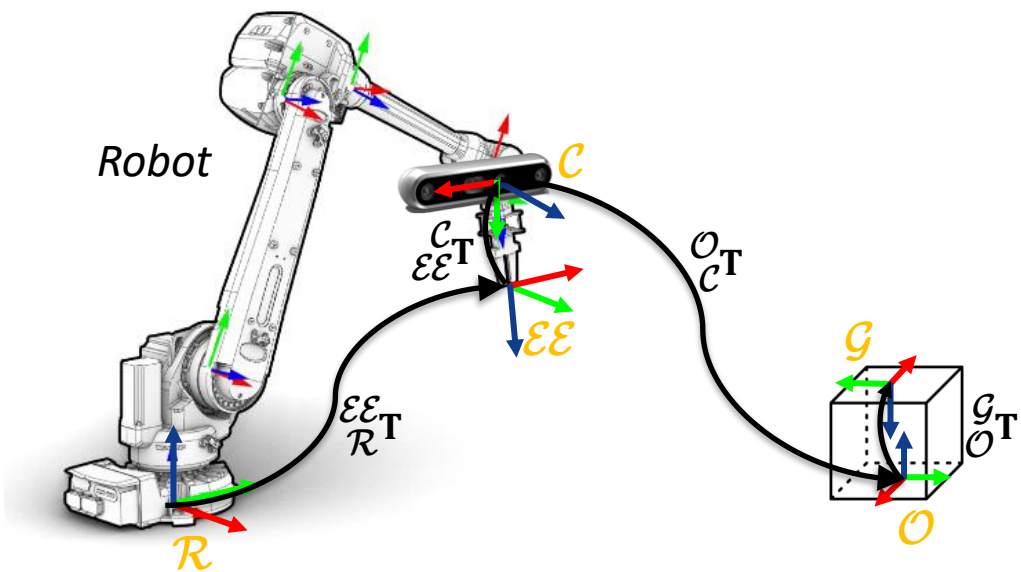
A predefined reference model of the object (e.g. CAD or previously scanned point cloud) is available with all point coordinates expressed in $\mathcal{O}$. Transform from object frame to grasp pose frame. Defined in the CAD model.



*To determine the current pose of the object in the **camera coordinate system** $\mathcal{C}$, the registration algorithm (eg., ICP) aligns the reference model to the observed point cloud captured by the camera. The output of ICP is the **rigid transformation** that maps points from the object frame $\mathcal{O}$ to the camera frame $\mathcal{C}$ - denoted as: ${}^{\mathcal{O}}_{\mathcal{C}}\mathbf{T}$*



Transformation chain to express the object or grasp pose in the robot's base frame



$\mathcal{R}$ - *robot base coordinate system*
$\mathcal{C}$ - *camera coordinate system*
$\mathcal{F}$ - *flange coordinate system*
$\mathcal{EE}$ - *end effector coordinate system*
$\mathcal{O}$ - *object coordinate system*
$\mathcal{G}$ - *grasp pose coordinate system*

$$ {}^{\mathcal{G}}_{\mathcal{R}}\mathbf{T} = {}^{\mathcal{EE}}_{\mathcal{R}}\mathbf{T}_1 \, {}^{\mathcal{C}}_{\mathcal{EE}}\mathbf{T} \, {}^{\mathcal{O}}_{\mathcal{C}}\mathbf{T}_1 \, {}^{\mathcal{G}}_{\mathcal{O}}\mathbf{T} $$

To execute the grasp, the end-effector must be positioned such that:

$$ {}^{\mathcal{EE}}_{\mathcal{R}}\mathbf{T} = {}^{\mathcal{G}}_{\mathcal{R}}\mathbf{T} $$

Limitations of classical registration in humanoid robotics household tasks

- ***Dependence on accurate object geometry –*** *ICP assumes a precise CAD or template model, which is not valid for objects like fruit where shape, size, and surface vary significantly within the same class, leading to poor or inconsistent pose estimation.*
- ***Sensitivity to clutter and partial overlap*** *– In household scenes with occlusions and touching objects, ICP may align to incorrect surfaces or mixed-object geometry, producing unstable or biased transformations.*
- ***Local minimum convergence*** *– ICP is a local optimizer; with incomplete views or repetitive geometry it can converge to wrong solutions despite low residual error.*
- ***Weak performance on textureless / symmetric objects*** *– Smooth or symmetric objects (e.g., apples, cups, bowls) provide limited geometric constraints, leading to ambiguous or non-unique 6DoF estimates.*
- ***Degradation with noisy or missing depth*** *– Reflective, transparent, or dark surfaces reduce depth quality, causing unreliable correspondences and inaccurate pose refinement.*
- ***Computational cost and real-time limitations*** *– Multi-view acquisition and iterative registration increase latency, making it difficult to achieve robust, real-time perception in dynamic household environments.*

Perception – object detection and pose estimation

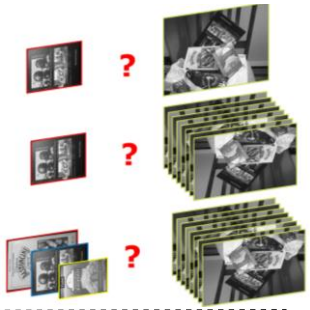
Object detection

Apple

Find an object in an image

Find an object in multiple images

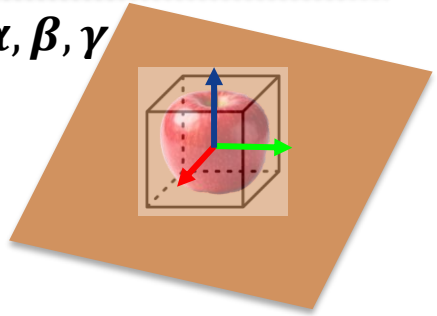
Find multiple objects in multiple images



Segmentation

2D pixels, 3D point cloud

$x, y, z, \alpha, \beta, \gamma$



6DOF Pose Estimation

2D–3D pipeline for robust object pose estimation in unstructured and cluttered environments

- ***2D-guided 3D registration*** → *instance segmentation constrains the point cloud, improving robustness to clutter, occlusions, and noise*
- ***Semantic priors + geometric refinement*** → *detection provides initialization, while local registration ensures accurate 6DoF pose*
- ***Suitability for humanoid robotics*** → *robust perception pipeline for unstructured household environments, enabling reliable manipulation of diverse objects*

Object detection - 2D + 3D

- ***1. Color-based 2D filtering and 3D point selection***
  The image is converted to HSV, thresholded with inRange, and the selected image pixels are mapped to the corresponding 3D points using the organized point cloud.
- ***2. Sphere detection with RANSAC***
  pcl::SACSegmentation with the sphere model uses RANSAC to find spherical objects in the filtered point cloud within a defined size range.
- ***3. 3D cluster detection and box description***
  pcl::EuclideanClusterExtraction separates the filtered cloud into individual object clusters, and MomentOfInertiaEstimation computes an oriented bounding box for each one.
- ***4. Cylinder detection with normals and RANSAC***
  Surface normals are estimated first, then pcl::SACSegmentationFromNormals fits a cylinder model using both point geometry and normal direction information.

2D color filtering and 3D point selection

- ***cv::cvtColor(...) — RGB/BGR to HSV conversion***
  Converts the input image from BGR to HSV color space so that color-based thresholding is easier and more robust.
- ***cv::inRange(...) — HSV thresholding***
  Creates a binary mask that keeps only image pixels whose HSV values lie inside the selected color range.
- ***cv::copyTo(..., mask) — masked 2D image extraction***
  Generates a filtered 2D image in which only the selected color region remains visible.
- ***organized point cloud indexing (organized_cloud->points[index]) — 2D to 3D mapping***
  Uses the organized cloud structure so that each selected image pixel directly retrieves the corresponding 3D point.
- ***valid-point check (is_valid_xyz(...)) — removal of invalid depth samples***
  Rejects 3D points with missing or non-finite coordinates before further processing.

PCL -> model fitting for grasping

Plane *→ table surface, floor, wall, cabinet door, shelf*

Cylinder *→ mug, water bottle, food can, glass, thermos, spice jar*

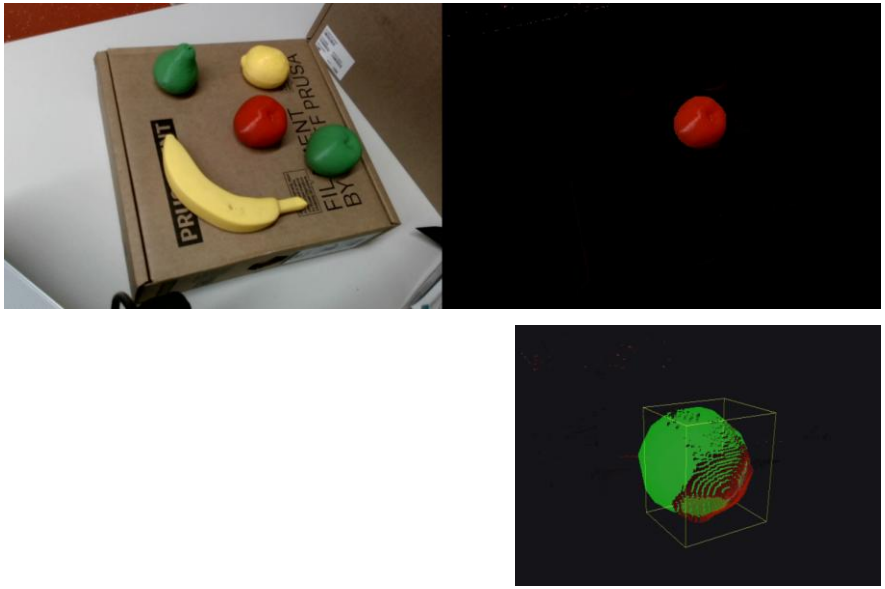
Sphere *→ apple, orange, ball, onion*

Prism *→ book, cereal box, milk carton, smartphone, sponge, storage box*

Sphere detection by RANSAC model fitting

- ***pcl::SACSegmentation<PointT> — sample-consensus model fitting setup***
  Configures a RANSAC-based geometric fitting procedure for detecting a sphere in the filtered 3D points.
- ***setModelType(pcl::SACMODEL_SPHERE) — sphere model selection***
  Specifies that the target geometric model is a sphere.
- ***setMethodType(pcl::SAC_RANSAC) — RANSAC estimation***
  Uses RANSAC to repeatedly test candidate sphere models and keep the one supported by the largest set of inlier points.
- ***setDistanceThreshold(...) — inlier distance criterion***
  Defines how far a point may be from the ideal sphere surface and still be counted as an inlier.
- ***setRadiusLimits(...) — sphere size constraint***
  Restricts detection to spheres whose radius lies inside the expected size range.
- ***segment(inliers, coefficients) — sphere estimation***
  Computes the sphere parameters and returns the set of 3D points that best fit the detected sphere.
- ***remove_points_by_indices(...) — iterative instance removal***
  Removes the inlier points of the detected sphere so that additional spheres can be searched in the remaining cloud.

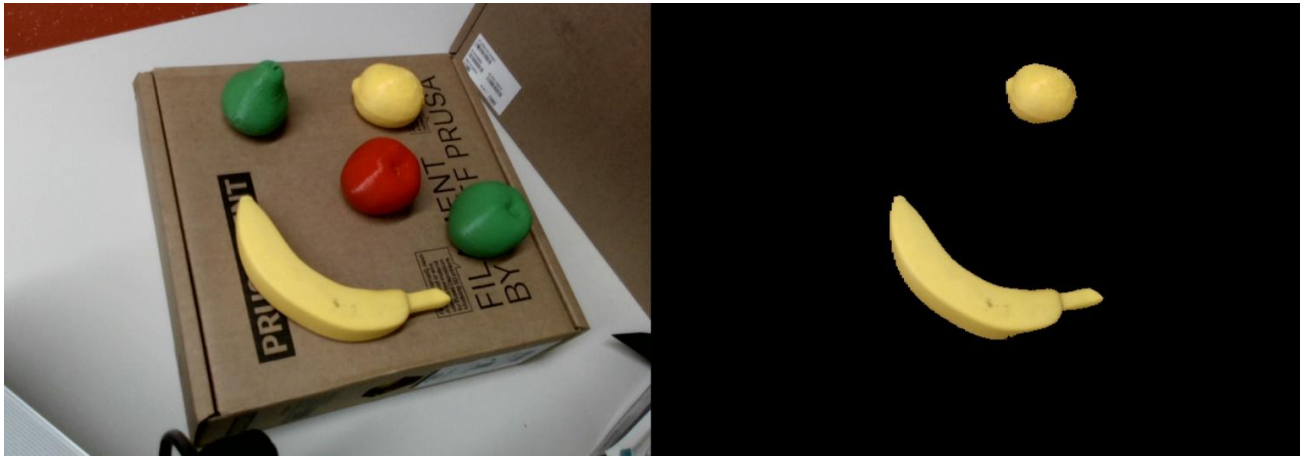
Object detection and pose estimation – sphere fitting

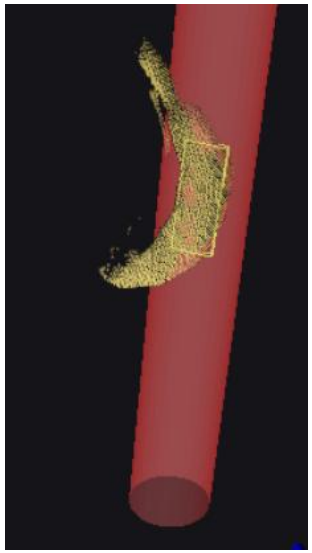


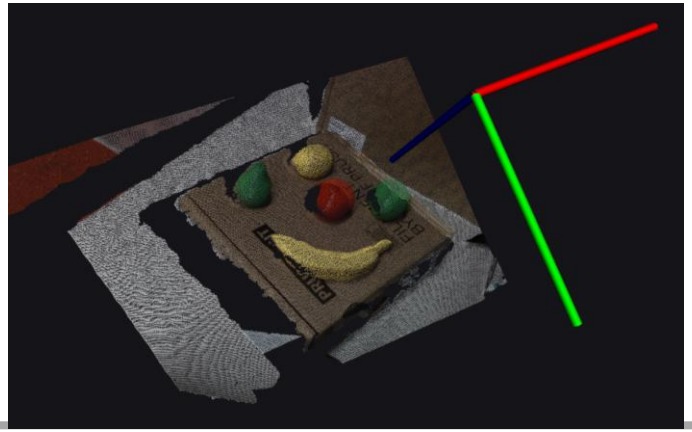
Cylinder Detection by Normals + RANSAC Model Fitting

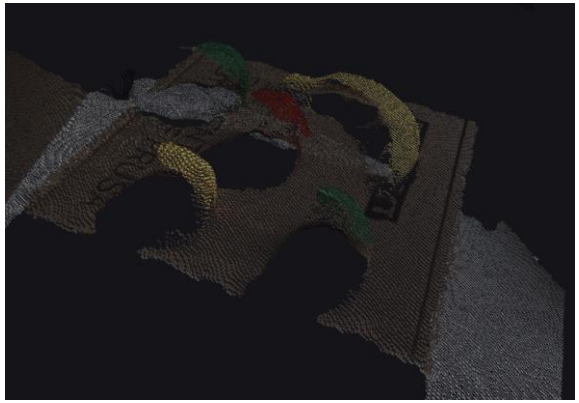
- ***pcl::NormalEstimation<PointT, pcl::Normal> — surface normal estimation***
  Computes a normal vector for each point so that local surface orientation can support cylinder fitting.
- ***setKSearch(...) — local neighborhood for normals***
  Defines how many neighboring points are used when estimating each surface normal.
- ***pcl::SACSegmentationFromNormals<PointT, pcl::Normal> — model fitting with geometric normals***
  Configures a RANSAC-based cylinder fitting process that uses both point positions and surface normals.
- ***setModelType(pcl::SACMODEL_CYLINDER) — cylinder model selection***
  Specifies that the target geometric model is a cylinder.
- ***setMethodType(pcl::SAC_RANSAC) — robust cylinder estimation***
  Uses RANSAC to test many cylinder candidates and select the one best supported by the data.
- ***setNormalDistanceWeight(...) — normal consistency weighting***
  Controls how strongly the estimated surface normals influence cylinder fitting in addition to point-to-model distance.
- ***setRadiusLimits(...) — cylinder size constraint***
  Restricts fitting to cylinders whose radius lies inside the expected diameter range.
- ***segment(inliers, coefficients) — cylinder estimation***
  Computes the cylinder axis and radius and returns the points that best support that cylinder model.
- ***pcl::MomentOfInertiaEstimation<PointT> / getOBB(...) — bounding box of cylinder inliers***
  Computes an oriented bounding box around the cylinder inlier points to describe the detected object's spatial extent.

Object detection and pose estimation – cylinder fitting









Main limitations – model fitting

High intra-class variability *– significant variation in size, shape, and color reduces robustness of fixed thresholds and models.*

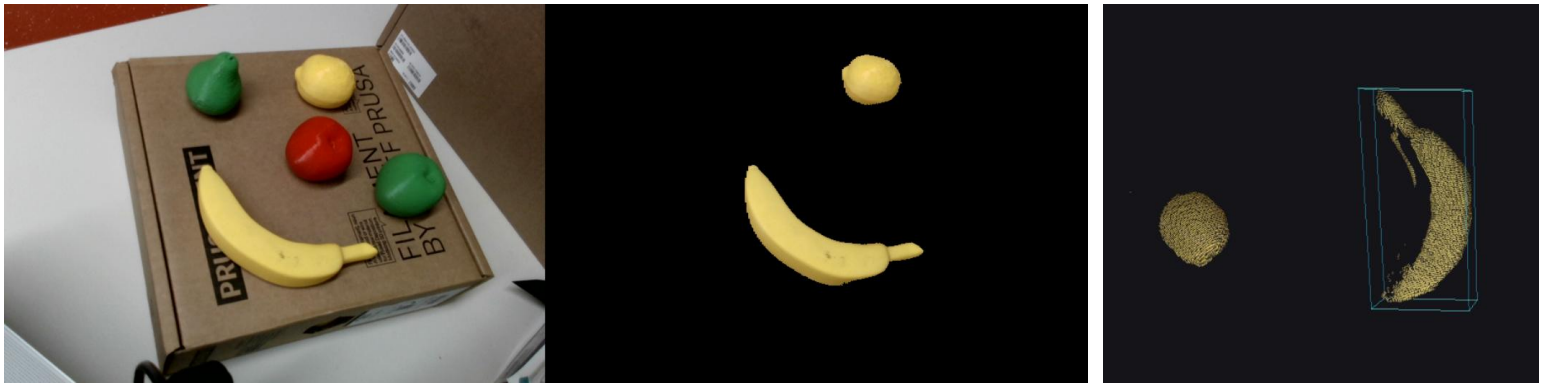
Sensitivity to illumination and color overlap *– hsv-based segmentation is affected by lighting changes and may fail for similar-colored objects.*

Geometric model mismatch *– real fruit deviates from ideal sphere/cylinder models, leading to approximate and sometimes incorrect fits.*

3D cluster detection and bounding box extraction

- ***pcl::search::KdTree<PointT> — neighborhood search structure***
  Builds a spatial search tree that allows efficient nearest-neighbor queries in the point cloud.
- ***pcl::EuclideanClusterExtraction<PointT> — Euclidean clustering***
  Groups nearby 3D points into separate clusters based on spatial distance.
- ***setClusterTolerance(...) — cluster gap threshold***
  Defines the maximum distance between neighboring points that may still belong to the same cluster.
- ***setMinClusterSize(...) — minimum cluster size filtering***
  Rejects very small point groups so that noise is not treated as a real object.
- ***extract(cluster_indices) — cluster segmentation***
  Returns the indices of all detected 3D point clusters.
- ***select_points_by_indices(...) — cluster point extraction***
  Builds an individual point cloud for each detected cluster.
- ***pcl::MomentOfInertiaEstimation<PointT> — oriented bounding box estimation***
  Computes an oriented bounding box that approximates the size and orientation of each detected 3D cluster.
- ***getOBB(...) — cluster size and pose description***
  Returns the box dimensions, center, and orientation used to describe the spatial extent of the cluster.

Object detection and pose estimation – cluster detection and bounding box extraction



Main limitations

Merged objects in clutter *– touching fruits are often segmented as a single region or cluster, preventing reliable instance separation.*

High intra-class variability *– significant variation in size, shape, and color reduces robustness of fixed thresholds and models.*

Sensitivity to illumination and color overlap *– hsv-based segmentation is affected by lighting changes and may fail for similar-colored objects.*

Geometric model mismatch *– real fruit deviates from ideal sphere/cylinder models, leading to approximate and sometimes incorrect fits.*

Clustering limitations at contact boundaries *– distance-based clustering fails when objects are physically touching or densely packed.*

Unreliable pose for manipulation in clutter *– estimated models may correspond to merged or partial objects, not to a graspable single fruit.*

Main limitations → potential solutions

Merged objects in clutter *→ top-down picking strategy, occlusion reasoning, contact-aware separation*

High intra-class variability *→ adaptive* ***CNN****-based 2D detection (fine-tuning, data augmentation)*

Sensitivity to illumination and color overlap → learned features (CNN)*, illumination-invariant preprocessing, domain adaptation*

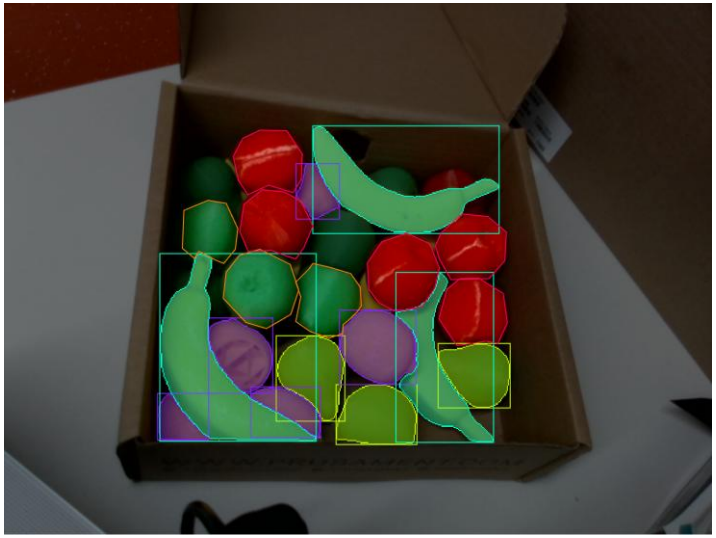
Geometric model mismatch → instance segmentation *(2D) + local 3D reconstruction / model-free fitting*

Clustering limitations at contact boundaries *→ multi-modal segmentation (RGB-D),* ***instance segmentation****, graph-based clustering*

Unreliable pose for manipulation in clutter → multi-view reconstruction *, grasp affordance detection, local surface analysis*

Object class detection + instance segmentation

- ***Object class detection localizes and classifies objects*** *(e.g., via bounding boxes using CNN-based detectors such as YOLO (object detection) or Faster R-CNN), providing coarse spatial priors that constrain subsequent 3D processing and reduce the search space for pose estimation.*
- ***Instance segmentation extends detection to pixel-level masks for each object instance*** *(e.g., Mask R-CNN or Segment Anything Model), enabling precise object isolation in clutter and direct extraction of object-specific point clouds for robust 6DoF pose estimation and grasp planning.*



Building a complete 3D scene from multiple views

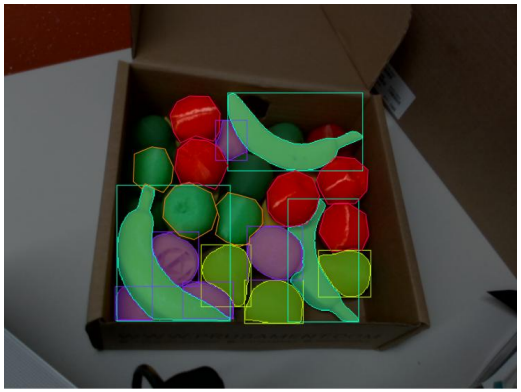
- ***Multi-view acquisition*** *– capture multiple RGB-D views of the scene by moving the camera or using an eye-in-hand robotic setup.*
- ***Known transformations*** *– use robot kinematics or calibrated camera poses to place each point cloud into a common reference frame.*
- ***Initial point cloud alignment*** *– transform each captured point cloud using the known poses to obtain a coarse global reconstruction.*
- ***Registration refinement*** *– apply iterative closest point (ICP) to reduce alignment errors and improve overlap between views.*
- ***Point cloud fusion*** *– merge aligned point clouds into a single, denser and more complete 3d representation.*
- ***Improved coverage and accuracy*** *– multi-view stitching reduces occlusions and missing regions, enabling more reliable detection and pose estimation.*

Student assignment - prompt

- *Modify, improve and use the following prompt and create your own object detection pipeline:*
  - *Use C++ with OpenCV for 2D image processing and PCL for synchronized 3D point-cloud processing from paired RGB image and `.pcd` data.*
  - *Load aligned image–cloud pairs from disk, match them by filename key, and verify that the point cloud is organized so each image pixel corresponds to one 3D point.*
  - *Convert the RGB image from BGR to HSV color space and apply `cv::inRange()` thresholding to create an interactive color-based segmentation mask.*
  - *Create OpenCV trackbars for dataset selection, HSV lower/upper bounds, and model-selection mode so the segmentation and fitting pipeline can be tuned live.*
  - *Use the binary mask to filter both the 2D image and the 3D cloud, keeping only pixels and corresponding 3D points that satisfy the HSV condition.*
  - *Display the original and filtered image side by side in OpenCV, with text overlays indicating the active method and current processing mode.*
  - *Visualize the filtered 3D result in a PCL visualizer using RGB point rendering, coordinate axes, and status text linked to the active image mask.*
  - *Select the geometric analysis method through a mode switch: no fitting, sphere fitting, Euclidean cluster extraction with oriented bounding boxes, or cylinder fitting.*
  - *For sphere and cylinder detection, use PCL RANSAC-based model fitting with distance constraints, inlier-ratio validation, and optional normal estimation for cylinder models.*
  - *For cluster-based object analysis, use KD-tree neighborhood search, Euclidean clustering, and moment-of-inertia-based oriented bounding boxes to estimate object extent and pose.*
  - *Output the estimated object centers (e.g., sphere center, cylinder axis point, or OBB centroid) to the terminal using formatted console logging for each detected instance.*

Fruit detection and pose estimation

- *The joint task is for students to divide among themselves 6 different fruit classes (3D printed) and perform annotation (**INSTANCE SEGMENTATION**) according to the lecture instructions and the dataset merging guidelines (see the additional PDF "Exercises"; pay attention to the last slides – "Dataset merging").*
- *First, create a shared .csv file with consistent class names (red apple, green apple, lemon, banana, avocado, walnut, pear). All students must upload this file to Roboflow to ensure consistent class labeling for CNN training.*
- *After each student independently annotates their own training set (without augmentations), all datasets must be merged into a single dataset. This merged dataset is then uploaded again to Roboflow, where augmentations are applied and the final dataset is used for training.*
- *Train a YOLOv26nano model, analyze the results, and submit the trained model and report*



The end